

Multithreaded Programming

Thread

A thread is the **smallest unit of execution inside a program**.

A Java program starts with one thread, called the Main Thread

```
public class Demo {  
    public static void main(String[] args) {  
        System.out.println("Hello");  
        System.out.println("Java");  
    }  
}
```

Here, only one thread executes both statements.

Multithreading

Multithreading is the process of executing **two or more threads simultaneously within a single program**. It allows multiple tasks to run concurrently.

Advantages

- Better CPU utilization
- Faster execution
- Multiple tasks simultaneously
- Responsive applications
- Resource sharing

Ways to Create Threads

Method 1: Extending Thread Class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
    public static void main(String args[]) {  
        MyThread t = new MyThread();  
        t.start();  
    }  
}
```

Thread is running

Note: If the class extends the **Thread** class, the thread can be run by creating an instance of the class and call its **start()** method.

Method 2: Implementing Runnable Interface

```
class MyRunnable implements Runnable {  
  
    public void run() {  
        System.out.println("Runnable Thread");  
    }  
  
    public static void main(String args[]) {  
  
        MyRunnable obj = new MyRunnable();  
  
        Thread t = new Thread(obj);  
  
        t.start();  
    }  
}  
Runnable Thread
```

Note: If the class implements the **Runnable** interface, the thread can be run by passing an instance of the class to a **Thread** object's constructor and then calling the thread's **start()** method.

```
class Pasta extends Thread {  
    public void run() {  
        System.out.println("Pasta is being prepared.");  
    }  
}  
  
class Salad extends Thread {  
    public void run() {  
        System.out.println("Salad is being prepared.");  
    }  
}  
  
public class Restaurant {  
    public static void main(String[] args) {  
  
        Pasta t1 = new Pasta();  
        Salad t2 = new Salad();  
  
        t1.start();  
        t2.start();  
    }  
}
```

Pasta is being prepared.
Salad is being prepared.

(The order may change because both threads run concurrently.)

```
class Cooking extends Thread {  
    String food;  
  
    Cooking(String food) {  
        this.food = food;  
    }  
  
    public void run() {  
        System.out.println(food + " is being prepared.");  
    }  
}
```

```
public class Restaurant {  
  
    public static void main(String[] args) {  
  
        Cooking t1 = new Cooking("Pasta");  
        Cooking t2 = new Cooking("Salad");  
        Cooking t3 = new Cooking("Rice");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

Rice is being prepared.
Pasta is being prepared.
Salad is being prepared.
(Output order is not fixed.)

```
class Cooking extends Thread {  
  
    String food;  
  
    Cooking(String food) {  
        this.food = food;  
    }  
  
    public void run() {  
        System.out.println(food + " prepared by " + getName());  
    }  
}
```

```
public class Restaurant {  
  
    public static void main(String[] args) {  
  
        Cooking t1 = new Cooking("Pasta");  
        Cooking t2 = new Cooking("Salad");
```

```
t1.start();
t2.start();
}
```

Pasta prepared by Thread-0
Salad prepared by Thread-1

Differences between "extending" and "implementing" Threads

The major difference is that when a class extends the Thread class, you cannot extend any other class, but by implementing the Runnable interface, it is possible to extend from another class as well.

Case 1: Extending Thread (Cannot Extend another Class)

```
class Animal {

    void sound() {
        System.out.println("Animal Sound");
    }
}
class Dog extends Animal, Thread { // "NOT ALLOWED IN JAVA"

    public void run() {
        System.out.println("Thread Running");
    }
}
```

Compilation Error

Case 2: Using Runnable (Allowed)

```
class Animal {

    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal implements Runnable { // ALLOWED

    public void run() {
        System.out.println("Thread Running");
    }

    public static void main(String args[]) {
        Dog d = new Dog();
        Thread t = new Thread(d);
        d.sound();
        t.start();
    }
}
```

}
Animal Sound
Thread Running

Concurrency Problems

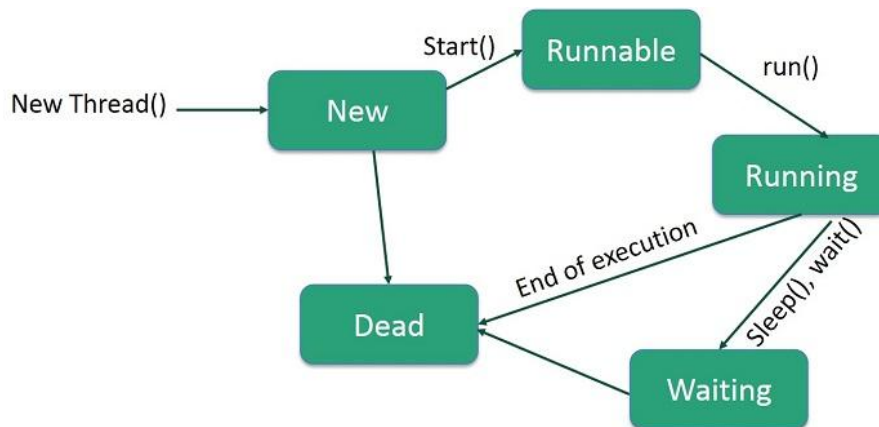
```
public class Main extends Thread {  
    public static int amount = 0;  
  
    public static void main(String[] args) {  
        Main obj = new Main();  
        obj.start();  
        System.out.println(amount);  
        amount++;  
        System.out.println(amount);  
    }  
  
    public void run() {  
        amount++;  
    }  
}  
0  
1  
OR  
0  
2  
OR  
1  
2
```

Use **isAlive()** to prevent concurrency problems

```
public class Main extends Thread {  
    public static int amount = 0;  
  
    public static void main(String[] args) {  
        Main obj = new Main();  
        obj.start();  
        // Wait for the thread to finish  
        while(obj.isAlive()) {  
            System.out.println("Waiting...");  
        }  
        System.out.println("Main: " + amount);  
        amount++;  
        System.out.println("Main: " + amount);  
    }  
    public void run() {  
        amount++;  
    }  
}  
Waiting...
```

Waiting...
Main: 1
Main: 2
OR
Waiting...
Main: 1
Main: 2

Life Cycle of a Thread



- **New** – A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.
- **Runnable** – after a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting** – Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.
- **Timed Waiting** – A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transition back to the runnable state when that **time interval** expires or when the event it is waiting for occurs.
- **Terminated (Dead)** – A runnable thread enters the terminated state when it completes its task or otherwise terminates.


```

class MyThread extends Thread {

    public void run() {

        System.out.println("Running State");

        try {
            System.out.println("Sleeping...");
            Thread.sleep(3000);    // Waiting State
        }
        catch(Exception e) { } //InterruptedException (checked exception)

        System.out.println("Thread Finished");
    }
}

public class Main {

    public static void main(String[] args) {

        MyThread t = new MyThread();

        // NEW State
        System.out.println("1. New State");

        // Runnable State
        t.start();
        System.out.println("2. Runnable State");

        while(t.isAlive()) {
            // Wait until thread finishes. Not a thread state
        }

        // Dead State
        System.out.println("3. Dead State");
    }
}
1. New State
2. Runnable State
Running State
Sleeping...
Thread Finished
3. Dead State
OR
1. New State
Running State
Sleeping...
2. Runnable State
Thread Finished
3. Dead State

```

Thread Priorities

Java thread priorities are in the range between `MIN_PRIORITY` (a constant of 1) and `MAX_PRIORITY` (a constant of 10). By default, every thread is given priority `NORM_PRIORITY` (a constant of 5).

Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.

However, thread priorities cannot guarantee the order in which threads execute and are very much platform dependent.